



Fundamentals of Programming: Arrays

Abdul Haseeb

Agenda

- Need of arrays
- Declaration and Initialization of arrays
- Accessing elements of an array
- Traversing an array using for loop and range based for loop
- Taking Input in array
- 2D Arrays

Problem

3



Problem:

Store the price of 4 to 5 different products in a store.



Solution:

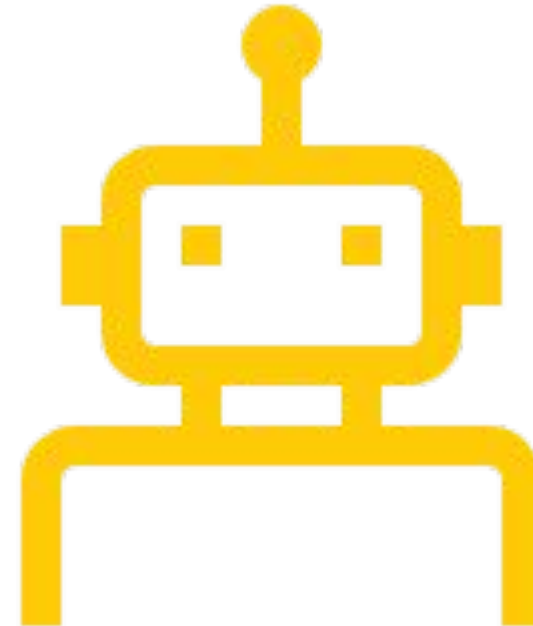
Create 5 different variables:

- float price1;
- float price2;
- float price3;
- float price4;
- float price5;

Another Problem

4

- ▶ Store ages of all family members
- ▶ Let's say you have to store the names of students in a class
- ▶ Now let's expand it to the whole university



Solution

- ▶ Arrays
 - ▶ Avoid the repetitive task of declaring and initializing the variable

```
int ages[] = {71, 42, 37, 5, 18};
```

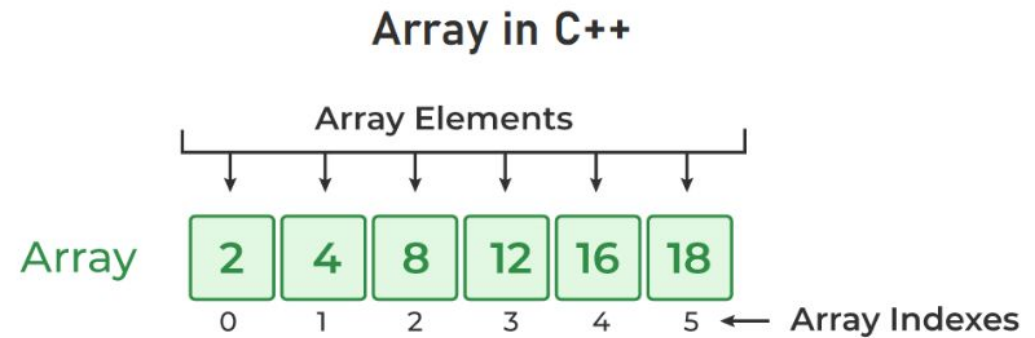
What is an Array?

- ▶ **Data Structure** that is used to store **multiple values** of **similar type** in a **contiguous memory** location
- ▶ An array is also called as homogenous collection of data
- ▶ **Examples:**
 - ▶ Marks of students in a class
 - ▶ Collection of medicines in a medical store

An array of 6 numbers

- ▶ Index of an array starts from 0 to $n-1$
- ▶ (where n is the size of an array)

7



Index

INDEX IS THE POSITION-1 WHERE AN ARRAY
ELEMENT IS STORED

Declaration of an array

- ▶ int is the data type, you can use char, float, double etc
- ▶ arr is the name of array variable, which follows naming rules of a variable
- ▶ 5 inside the square brackets refers to the size of array, which can not be changed

```
data_type array_name[Size_of_array];
```

Example

```
int arr[5];
```

What do you
guess how
much bytes in
memory an
array will
occupy?

- ▶ It depends on the data type and size of an array

Initialization of an array

MANY POSSIBLE WAYS LET'S DISCUSS SOME
COMMON WAYS

1. Initialize Array with Values and size in C++

```
int arr[5] = {1, 2, 3, 4, 5};
```

1. Size of array is fixed in this case, can not exceed 5 elements

2. Initialize Array with Values and without Size in C++

```
int arr[] = {1, 2, 3, 4, 5};
```

1. Size of array is decided based on the number of elements inside curly braces

```
3
4 int main(){
5
6
7 int nums[5];
8
9 nums[0]=3;
10 nums[1]=4;
11 nums[2]=4;
12 nums[3]=4;
13 nums[4]=4;
```

3. Initialize
with the
help of
indexes

Creating an array of names

FQ JIXDYMEBSLJBWXDUNL
GFBVWLC TFP OIZQAYWHAT
MYVLOYFJR CVUNIJ PNJHI
WZUXQURAXIOMVMVOFTDC
VYCDYCJ K MOPX EFR SPCOB
KBJ IMUKIVAGVGRQNT EZH
ZHYBSECNIMDGOMFVETO E
CIPUYKFIXOCTFZCHJEAR
YKRVEGICRLXCLKLCTRD
QLGZRWF PFOEIYFVRMZHX
RPZYDUIVTEAXLJWSIRUG
JLAVMPLOTYCKIBQYWYPK
BPF RDJTVAQIFSTZVFMJC
SYECVINGFBRNYUCBSNTD
CFIBRMSZJEDXRWTKADFE

Creating
array of
vowel
letters

4. Initializing all the elements of an array with zero

- ▶ This method only works for zero

```
int zero_array[5] = {0};
```

Accessing an element of array

- ▶ Element can be accessed with the help of:
 - ▶ Array name
 - ▶ Along with an index number specified in the square braces

Example: `cout<<nums[3];` //It will print 4th element of the nums array on console screen


```
// C++ Program to Illustrate How to Access Array Elements
#include <iostream>
using namespace std;

int main()
{
    int arr[3];

    // Inserting elements in an array
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;

    // Accessing and printing elements of the array
    cout << "arr[0]: " << arr[0] << endl;
    cout << "arr[1]: " << arr[1] << endl;
    cout << "arr[2]: " << arr[2] << endl;

    return 0;
}
```

Example of array element access

Traversing an array

- ▶ Traversing means to visit each element of array exactly once

```
// C++ Program to Illustrate How to Traverse an Array
#include <iostream>
using namespace std;

int main()
{
    // Initialize the array
    int table_of_two[10]
        = { 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 };

    // Traverse the array using for loop
    for (int i = 0; i < 10; i++) {
        // Print the array elements using indexing
        cout << table_of_two[i] << " ";
    }

    return 0;
}
```

Exercise: Must complete before coming tomorrow

21

- ▶ Declare and Initialize an array of 6 integers, namely `nums_array`
- ▶ Print all the values of array without loop, with the help of array index
- ▶ Print all the values of array through for loop, with the help of array index

```
string friends[] = {"Alan", "Bob", "Carol", "David", "Ellen",  
                  "Fred", "Grace", "Henry", "Ian", "Jen"};  
  
for (string i : friends) {  
    cout << i << endl;  
}
```

Traversing an array using enhanced for loop

```
string friends[] = {"Alan", "Bob", "Carol", "David", "Ellen",  
                   "Fred", "Grace", "Henry", "Ian", "Jen"};  
  
cout << sizeof(friends) / sizeof(friends[0]) << endl;
```

Example of using sizeof operator to find length of an array

what to do for a value other than zero to assign same value to all elements of an array?

24

- ▶ What about for loop?



```
for (int i = 0; i < N; i++) {  
    arr[i] = value;  
}
```

5. Initializing with the help of loop

```
arr[i] = new_value;
```

Update array element

Checking the size of an array and array element using sizeof()

27

```
#include <iostream>
using namespace std;

int main()
{
    int arr[] = { 1, 2, 3, 4, 5 };

    // Size of one element of an array
    cout << "Size of arr[0]: " << sizeof(arr[0]) << endl;

    // Size of array 'arr'
    cout << "Size of arr: " << sizeof(arr) << endl;

    // Length of an array
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Length of an array: " << n << endl;

    return 0;
}
```

Access the memory address of an array using address(&) operator

```
// Printing address of the array using array name  
cout << "Memory address of arr: " << &arr << endl;
```



```
1  #include<iostream>
2  using namespace std;
3
4  int main(){
5
6  int arr[5];
7
8  cout<<&arr<<endl;
9  cout<<&arr[0]<<endl;
10 cout<<&arr[1]<<endl;
11 cout<<&arr[2];
12 }
13
```

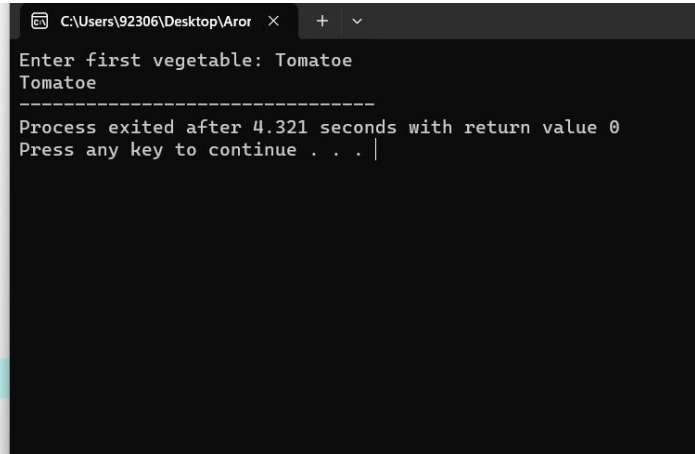
& Operator
returns the
starting address of
an array element


```
#include<iostream>
using namespace std;

int main(){

string vegies[5];

cout<<"Enter first vegetable: ";
cin>>vegies[0];
cout<<vegies[0];
```



```
C:\Users\92306\Desktop\Aror  x  +  v
Enter first vegetable: Tomatoe
Tomatoe
-----
Process exited after 4.321 seconds with return value 0
Press any key to continue . . . |
```

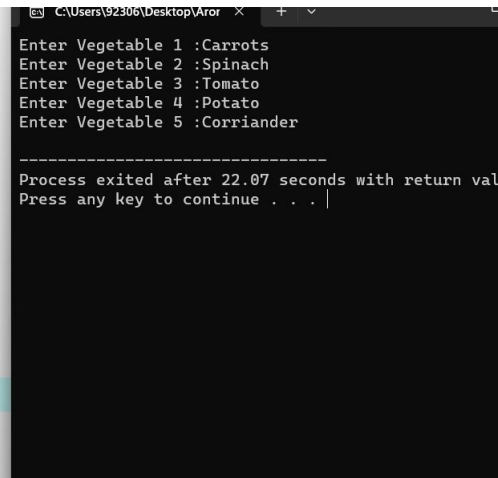
Taking Input inside an array

```
#include<iostream>
using namespace std;

int main(){

string vegies[5];
int array_length=sizeof(vegies)/sizeof(vegies[0]);
for(int i=0; i<array_length; i++){
    cout<<"Enter Vegetable "<<i+1<<" :";
    cin>>vegies[i];
}

return 0;
```

A screenshot of a C++ program running in a terminal window. The program prompts the user to enter five vegetables. The inputs are: Carrots, Spinach, Tomato, Potato, and Corriander. The program then displays a message indicating it has exited after 22.07 seconds and asks the user to press any key to continue.

```
C:\Users\92306\Desktop\Aror x + v
Enter Vegetable 1 :Carrots
Enter Vegetable 2 :Spinach
Enter Vegetable 3 :Tomato
Enter Vegetable 4 :Potato
Enter Vegetable 5 :Corriander

-----
Process exited after 22.07 seconds with return value 0
Press any key to continue . . .
```

Taking Multiple Inputs in an array

Activity

- ▶ Declare an array to store the weight of 5 persons
- ▶ Input the weight of 5 persons from the user
- ▶ Find out the average weight value (average weight will be sum of all the weights divided by number of persons)

Task

33

- ▶ Declare an array of 5 Integers
- ▶ Take input using loop
- ▶ Find out the maximum number from the array, using another loop



Two Dimensional Arrays

Matrix

35

- ▶ Set of number arranged in rows and columns
- ▶ Numbers are called elements of matrix
- ▶ An Image is nothing but a matrix
- ▶ If we say $m \times n$ matrix, then that means m rows and n columns
- ▶ 2×3 matrix:

$$\begin{bmatrix} 1 & 3 & 8 \\ 2 & -4 & 5 \end{bmatrix}$$

2D Array

36

- ▶ An Array inside another array
- ▶ Just like a table/matrix where there are rows and columns

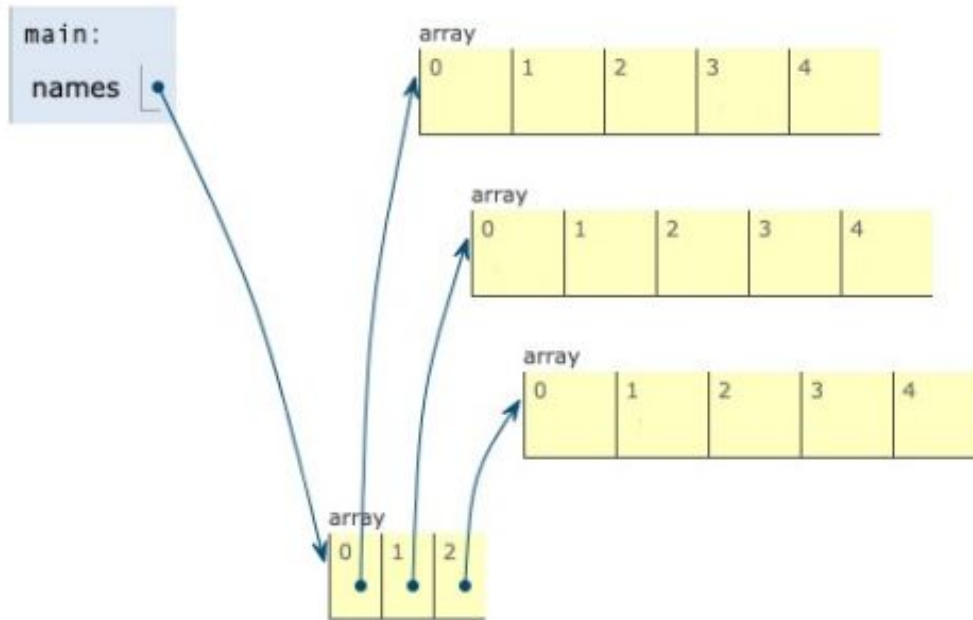
		Columns				
		0	1	2	3	4
Rows	0	Alan	Bob	Carol	David	Ellen
	1	Fred	Grace	Henry	Ian	Jen
	2	Kelly	Liam	Mary	Nancy	Owen

2D Array Syntax

- Array type followed by a name for the 2D array followed by **two** empty pairs of brackets [].
- The number of **rows** goes inside the **first** pair of brackets and the number of **columns** goes inside the **second** pair of brackets.

```
string names[3][5];
```

2D Array Syntax



Visualization of 2D Array

```
string names[3][5];  
  
cout << sizeof(names) / sizeof(names[0]) << " rows" << endl;  
cout << sizeof(names[0]) / sizeof(string) << " columns" << endl;
```

Using sizeof() with 2D Array to find number of rows and columns

```
string names[ ][5] = { {"Alan", "Bob", "Carol", "David",  
    "Ellen"},  
    {"Fred", "Grace", "Henry", "Ian", "Jen"},  
    {"Kelly", "Liam", "Mary", "Nancy",  
    "Owen"} };  
  
cout << names[1][2] << endl;
```

Initializing and Accessing 2D Arrays

```
string names[3][5] = { {"Alan", "Bob", "Carol", "David",  
                        "Ellen"},  
                      {"Fred", "Grace", "Henry", "Ian", "Jen"},  
                      {"Kelly", "Liam", "Mary", "Nancy",  
                        "Owen"} };  
  
cout << names[1][2] << endl;  
  
names[1][2] = "Harry";  
cout << names[1][2] << endl;
```

Update/Modification in 2D Arrays


```
int digits[3][3] = { {1, 2, 3},  
                     {4, 5, 6},  
                     {7, 8, 9} };  
  
int row = sizeof(digits) / sizeof(digits[0]); //number of rows  
int col = sizeof(digits[0]) / sizeof(int); // number of columns  
  
for (int i = 0; i < row; i++) {  
    for (int j = 0; j < col; j++) {  
        cout << digits[i][j] << endl;  
    }  
}
```

Traversing 2D Array using for loop

Traversing 2D Array using enhanced for loop

```
// Driver code
int main()
{
    const int i = 3, j = 3;
    int matrix[i][j] = {{1, 2, 3},
                        {4, 5, 6},
                        {7, 8, 9}};

    for (auto &row : matrix)
    {
        for (auto &column : row)
        {
            cout << column << " ";
        }
        cout << endl;
    }
}
```

Task

44

- ▶ Write C++ code to sum the even numbers in the following array:

11	15	20
23	24	26
10	20	33

Future topics

45



ARRAYS AND
FUNCTIONS



ARRAYS AND
POINTERS